Prompts for Project 3

this as Prompt 0 in Antigravity)

Prompt 0 — Project rules & constraints Build a full-stack web app named **Vacations (Project 3)**.

Tech (must):

- Client: **React + TypeScript**
- State: **Redux**
- Server: **Node.js + Express**
- DB: **MySQL**
- Real-time: **socket.io**
- Styling: **Bootstrap or Material UI**
- Architecture: clean separation (routes/controllers → services/logic → DAL/db → models/types), like we learned in class.

Hard requirements:

- Folder root must be: Vacation-STUDENT_ID 27880 - React Nodejs and MySQL-…
- All uploaded images must be saved to: 27880 - React Nodejs and MySQL-… upload` (root-level upload folder) 27880 - React Nodejs and MySQL-…
- Entities:
- User: id, firstName, las27880 - React Nodejs and MySQL-… word
- Admin: single user (role-based)
- Vacation: id, description, destination, image, fromDate, toDate, price, followersCount 27880 - React Nodejs and MySQL-…
- Auth:
- Login + Register
- Registe27880 - React Nodejs and MySQL-… elds mandatory; username must be unique 27880 - React Nodejs and MySQL-…
- Permissions:
- Admin: add/edit/delet27880 - React Nodejs and MySQL-… orts
- User: view vacations, follow/unfollow 27880 - React Nodejs and MySQL-…
- Vacations page ordering:
- vacation27880 - React Nodejs and MySQL-… 2) then vacations not followed
- secondary sort by vacation date 27880 - React Nodejs and MySQL-…
- Real-time:
- If admin changes a vaca27880 - React Nodejs and MySQL-… w it see changes **without refresh** (socket.io) 27880 - React Nodejs and MySQL-…
- Admin reports:
- Use react-charts 27880 - React Nodejs and MySQL-… the link in doc)

- X axis: vacation name/destination
- Y axis: number of followers
- Only vacations with followers appear 27880 - React Nodejs and MySQL-…

Deliverables:

- MySQL schema + seed27880 - React Nodejs and MySQL-… files
- Scripts to run client and server locally

Now proceed step-by-step and do not skip testing.

1) Workspace & repository structure

Prompt 1 — Create the folder structure Create a monorepo folder named Vacation-STUDENT_ID with:

- backend/ (Node + Express)
- frontend/ (React + TS)
- upload/ (root-level folder for images)
- database/ (SQL schema + seed)
- README.md (how to run) Also add .gitignore (ignore node_modules, dist/build, .env, upload contents if needed but keep folder).

2) Database (MySQL) schema + seed

Prompt 2 — Design MySQL schema (no duplicates, proper relations) Inside database/ create:

- schema.sql that creates a database (name it vacations_db) and tables:
- users (id PK, firstName, lastName, username UNIQUE, passwordHash, role ENUM('admin','user'), createdAt)
- vacations (id PK, description, destination, imageName, fromDate, toDate, price DECIMAL, createdAt)
- followers (userId FK → users.id, vacationId FK → vacations.id, composite PK (userId, vacationId))
- Add indexes (username, vacationId, userId) and cascade rules. Also create seed.sql:
- Insert 1 admin user and several normal users
- Insert 8–12 vacations (some future dates, some near-future)
- Insert follower rows to simulate followers counts. Return both SQL files content.

Note: followersCount must be computed via query (or a view) rather than stored redundantly, unless you keep it consistent via queries.

3) Backend setup (Express + MySQL + auth)

Prompt 3 — Backend boilerplate Inside backend/:

- Initialize Node project

- Install: express, cors, mysql2, dotenv, jsonwebtoken, bcrypt, multer, socket.io, express-rate-limit (optional), joi/zod (validation), morgan (optional)
- Add TypeScript if you prefer backend TS; otherwise keep backend JS with strong structure (either is acceptable, but frontend is TS mandatory). Create:
- src/app.ts (or app.js) express app
- src/server.ts (or index.js) start server + socket.io
- .env.example with DB creds + JWT secret + port
- Central error handler middleware
- CORS enabled for frontend origin
- Health route GET /api/health

Prompt 4 — DB layer (mysql2 pool) + config Create a clean DB module:
- Reads env vars (host/user/password/database)
- Exposes execute(query, params) helper and transaction helper
- Uses mysql2/promise pool Add a simple repository/DAL pattern.

Prompt 5 — Auth: register/login + JWT + role middleware Implement endpoints:
- POST /api/auth/register
- requires firstName, lastName, username, password
- validates mandatory fields
- checks username unique
- hashes password with bcrypt
- creates role=user
- returns JWT + user payload (id, name, username, role)
- POST /api/auth/login
- validates username/password
- returns JWT + user payload Add middleware:
- verifyToken
- requireAdmin
- requireUser (optional) Make sure admin is "single admin" from seed (role=admin). 27880 - React Nodejs and MySQL-…

4) Backend: vacations CRUD + up

27880 - React Nodejs and MySQL-…

rts

Prompt 6 — Vacations REST API Create routes/controllers/services for vacations:
User routes:
- GET /api/vacations Returns vacations list with:
- followerCount

- isFollowedByMe (based on token user) Must be sortable on server OR return enough to sort on client.
- POST /api/follows/:vacationId (follow)
- DELETE /api/follows/:vacationId (unfollow)

Admin routes:

- POST /api/admin/vacations Create vacation with multipart/form-data:
- description, destination, fromDate, toDate, price, image(file) Save image to /upload in project root (not inside backend). 27880 - React Nodejs and MySQL-…
- PUT /api/admin/vacations/:id Edi27880 - React Nodejs and MySQL-… lly replace image (delete old file if replaced).
- DELETE /api/admin/vacations/:id Delete vacation + delete its followers + delete its image file.

Validation rules:

- price > 0
- fromDate <= toDate
- description/destination required Return consistent JSON and HTTP codes.

Prompt 7 — Admin report endpoint (followers per vacation) Create:

- GET /api/admin/reports/followers Returns only vacations with followers > 0, including:
- vacationId
- destination (or “name”)
- followersCount This will feed the react-charts column chart. 27880 - React Nodejs and MySQL-…

5) Socket.io real-time updates

27880 - React Nodejs and MySQL-…

io events design + server implementation** Implement socket.io so that:

- When a user logs in and opens vacations page, the client emits: join-user with userId (or token).
- When a user follows/unfollows, optionally join/leave a room per vacation: vacation:{id}
- When admin adds/edits/deletes a vacation:
- Emit events:
- vacation-added (to all users)
- vacation-updated (to room vacation:{id} AND also to users list so ordering updates)
- vacation-deleted (to all; and remove from rooms) Goal: users who follow a vacation see updates without refresh. 27880 - React Nodejs and MySQL-…

Also ensure server doesn't crash if soc

6) Frontend setup (React + TS + Redux + Router)

Prompt 9 — Frontend boilerplate Inside frontend/:

- Create React + TypeScript app
- Install: react-router-dom, redux toolkit, react-redux, axios, socket.io-client, bootstrap OR @mui/material, react-hook-form (optional), yup/zod (optional) Create folder structure:
- src/app/store.ts
- src/features/auth/
- src/features/vacations/
- src/features/reports/
- src/services/api.ts (axios instance)
- src/services/socket.ts
- src/routes/
- src/components/
- src/pages/ Add a ProtectedRoute component and an AdminRoute component.

7) UI screens (Login, Register, Vacations, Admin Manage, Admin Reports)

Prompt 10 — Login page Build /login:

- username + password
- on success: save JWT, update Redux auth slice, redirect to /vacations
- link to /register If user is not logged in and tries any route, redirect to /login.

Prompt 11 — Register page Bu

rstName, lastName, username, password

- all mandatory
- show server error if username exists
- on success: auto-login and redirect to /vacations 27880 - React Nodejs and MySQL-…

Prompt 12 — Vacations page (user

s) Build /vacations:

- Fetch GET /api/vacations with token
- Render cards

- Each card shows: image, destination, description, dates, price, followers count
- Follow/unfollow toggle button
- Ordering required:

1. followed vacations first
2. then not followed
3. inside each group: sort by fromDate (ascending) 27880 - React Nodejs and MySQL-… Add real-time updates:

- Connect socket.27880 - React Nodejs and MySQL-… r auth)
- Listen to events: added/updated/deleted
- Update Redux vacations list accordingly without refresh 27880 - React Nodejs and MySQL-…

**Prompt 13 — Admin main page (CRU

27880 - React Nodejs and MySQL-…

vacations` visible only to admin:

- Top button “Add vacation”
- Add flow: modal or separate page /admin/vacations/new (your choice)
- Each vacation card shows edit pencil + delete X only for admin 27880 - React Nodejs and MySQL-…
- Forms support image upload (multipart/27880 - React Nodejs and MySQL-… d/edit/delete:
- Update UI immediately
- Trigger socket.io events so users get real-time updates

Prompt 14 — Admin reports page (react-charts column) Build /admin/reports admin-only:

- Fetch GET /api/admin/reports/followers
- Use react-charts “column” example style (from doc link)
- X axis: destination/name
- Y axis: followersCount
- Only show vacations with followersCount > 0 27880 - React Nodejs and MySQL-…

8) Redux slices & services

P

27880 - React Nodejs and MySQL-…

kit slices Create Redux slices:

- authSlice: token, user, isLoggedIn; actions loginSuccess, logout
- vacationsSlice:

- list: Vacation[]
- actions: setVacations, followOptimistic, unfollowOptimistic, vacationAdded, vacationUpdated, vacationDeleted
- selectors for ordered list (followed first, then by date)
- reportsSlice for admin report data Also create typed hooks: useAppDispatch, useAppSelector.

Prompt 16 — API service + interceptors Create api.ts axios instance:
- baseURL from env
- request interceptor attaches JWT
- response interceptor handles 401 → logout → redirect to login

Prompt 17 — Socket service Create socket.ts:
- Connect only when token exists
- Expose connectSocket(token) and disconnectSocket()
- Register listeners that dispatch Redux actions
- Make sure you don't double-connect on re-render

9) Static file hosting for images

Prompt 18 — Serve /upload images correctly Backend must:
- Serve static: GET /upload/<filename> from project-root upload/ folder 27880 - React Nodejs and MySQL-… Frontend must:
- Display images using ba27880 - React Nodejs and MySQL-… ${imageName}`

10) Run scripts + README

Prompt 19 — Scripts and README Add root README.md with:
- Prereqs (Node, MySQL)
- How to create DB (run schema.sql + seed.sql)
- How to run backend (npm i, env, npm start/npm run dev)
- How to run frontend (npm i, npm start)
- Ports and env examples
- Admin credentials from seed Also add optional root script:
- npm run dev concurrently runs backend + frontend (optional)

11) QA / Testing prompts (what you asked to add)

Use these after the project works.

A) Backend tests

Prompt 20 — Backend unit + integration tests (Jest + Supertest) Add to backend:

- Jest config
- Supertest integration tests Create tests:

1. POST /api/auth/register
- success
- fails if missing field
- fails if username already exists

2. POST /api/auth/login
- success
- fails on wrong password

3. GET /api/vacations
- requires token

4. follow/unfollow endpoints
- updates follower count correctly

5. admin CRUD routes
- forbidden for normal user
- allowed for admin Use a test database or transactions rollback strategy.

B) Frontend tests

Prompt 21 — Frontend component tests (React Testing Library) Add tests:

- Login form renders + validates
- Register form renders + validates
- Vacations list sorting:
- followed vacations appear first
- date sorting inside group works
- Follow button dispatches and updates UI (mock API)
- Admin routes are blocked for non-admin

C) End-to-end tests

Prompt 22 — E2E tests (Cypress) Add Cypress tests:

1. Register → redirect to vacations
2. Login as user → follow a vacation → see it move to “followed section”
3. Login as admin → edit a followed vacation
4. Verify user client updates **without refresh** (socket.io real-time) 27880 - React Nodejs and MySQL-…
5. Admin report shows only vacations wit27880 - React Nodejs and MySQL-… D) QA checklist automation (full project “health”)

Prompt 23 — Full QA runner script Add a root script qa that runs:

- backend lint (if exists) + backend tests
- frontend lint (if exists) + frontend tests

- TypeScript typecheck
- Cypress headless e2e Output a clear summary at the end.

12) Bonus (Hosting)

Prompt 24 — Bonus hosting plan (optional) Give 2 deployment options:

1. Deploy backend (Node) + MySQL on a VPS or a platform that supports MySQL
2. Deploy frontend separately (static hosting) and configure CORS Include environment variables and how to handle /upload images (persistent storage). (Do not implement until local works perfectly.)